UNIT 17 90 min

pandas: Tables You Can Question

Load a table, inspect it, select and filter rows, create columns, handle missing values, and summarize by group.

By the end of this unit you will be able to

- Create a DataFrame and explain how it differs from a NumPy array
- Load a CSV with `pd.read_csv` and inspect it with `head`, `info`, and `describe`
- Select columns and rows with `[]`, `.loc`, and `.iloc`
- Filter rows with conditions and create new columns
- Find, count, and fill missing values
- Summarize with `groupby` and `value_counts`

What pandas adds

A **DataFrame** is a table: rows and columns, where each column has a name and can have its own type. A **Series** is one column. pandas builds on NumPy, so the speed is similar, and it adds labels, missing-value handling, and the reading and grouping tools that real data needs.

first_frame.py

```
import pandas as pd

df = pd.DataFrame({
    "name": ["Ana", "Ben", "Cy"],
    "score": [91, 84, 77],
    "period": [1, 2, 1]
})
print(df)
print(df["score"])
print(type(df["score"]).__name__)
```

> **Note**
> The convention is `import pandas as pd`.

Reading data

`pd.read_csv` loads a CSV file into a DataFrame. It accepts a file path or a URL. Here it reads from text held in memory, so the example runs in the sandbox. With a real file, pass its path instead.

read_csv.py

```
import io
import pandas as pd

csv_text = """name,period,score,absent
Ana,1,91,0
Ben,2,84,2
Cy,1,77,
Dee,2,95,1
"""
df = pd.read_csv(io.StringIO(csv_text))
print(df)
```

Inspecting a table

Call	What you learn
`df.head(3)`	The first rows, to see the layout
`df.shape`	(rows, columns)
`df.info()`	Column names, types, and how many values are missing
`df.describe()`	Count, mean, spread, and quartiles of the numeric columns
`df.columns`	The column names

inspect.py

```
import io
import pandas as pd

df = pd.read_csv(io.StringIO("name,period,score,absent\nAna,1,91,0\nBen,2,84,2\
print(df.shape)
print(df.columns.tolist())
print(df.describe())
```

Selecting: columns, then rows

Select columns with a list of names: `df[["name", "score"]]`. For rows, use `.loc` with labels, or `.iloc` with positions. `.iloc[0]` is the first row; `.loc[0]` is the row labeled 0. With the default index these are the same, but they stop matching once you sort or filter.

select.py

```
import pandas as pd

df = pd.DataFrame({"name": ["Ana", "Ben", "Cy"], "score": [91, 84, 77]})
print(df[["name", "score"]])
print(df.iloc[1])
print(df.loc[2, "score"])
print(df.iloc[0:2, 0])
```

Filtering rows

A condition on a column gives a True/False Series. Put it inside `df[...]` to keep the matching rows. Combine conditions with `&` and `|`, and put each one in parentheses.

filter.py

```
import pandas as pd

df = pd.DataFrame({"name": ["Ana", "Ben", "Cy", "Dee"],
                   "score": [91, 84, 77, 95],
                   "period": [1, 2, 1, 2]})
high = df[df["score"] >= 90]
print(high)
print(df[(df["period"] == 1) & (df["score"] < 90)])
```

New columns and sorting

columns_sort.py

```
import pandas as pd

df = pd.DataFrame({"name": ["Ana", "Ben", "Cy"], "score": [91, 84, 77]})
df["passed"] = df["score"] >= 80
df["curve"] = df["score"] + 5
print(df.sort_values("score", ascending=False))
```

Arithmetic on a column applies to every row, the same way it does in NumPy. Methods like `sort_values` return a new DataFrame, so assign the result if you want to keep it.

Missing values

Missing values appear as `NaN` (not a number). Count them with `isna().sum()`. Remove rows with `dropna()`, or fill them with `fillna()`. Decide which one before you summarize: a mean that silently skips missing values can hide a problem.

missing.py

```
import io
import pandas as pd

df = pd.read_csv(io.StringIO("name,absent\nAna,0\nBen,2\nCy,\n"))
print(df.isna().sum())
print(df.dropna())
print(df.fillna(0))
```

Watch out

An empty cell becomes `NaN`, which makes an integer column into a float column. That is normal, but check `info()` when a count column looks wrong.

groupby: summaries by category

`groupby(column)` splits the table into one group per value, then applies a summary to each group. The pattern is `df.groupby("period")["score"].mean()`: group by period, then take the mean score in each.

groupby.py

```
import pandas as pd

df = pd.DataFrame({
    "name": ["Ana", "Ben", "Cy", "Dee", "Eli"],
    "period": [1, 2, 1, 2, 1],
    "score": [91, 84, 77, 95, 88]
})
print(df.groupby("period")["score"].mean())
print(df.groupby("period")["score"].agg(["count", "min", "max"]))
print(df["period"].value_counts())
```

Tip

Use `agg` when you want several summaries at once. Use `value_counts()` to count categories in one column.

Predict the output THINK FIRST

Think about the shape and index of each result before you run it.

```
import pandas as pd
print(pd.DataFrame({"a": [1, 2]}).shape)
```

Answer: `(2, 1)`

```
import pandas as pd
df = pd.DataFrame({"x": [5, 6, 7]})
print(df[df["x"] > 5]["x"].tolist())
```

Answer: `[6, 7]`

```
import pandas as pd
df = pd.DataFrame({"x": [5, 6, 7]})
print(df.iloc[-1, 0])
```

Answer: `7`

```
import pandas as pd
print(pd.Series([1, None, 3]).isna().sum())
```

Answer: `1`

```
import pandas as pd
df = pd.DataFrame({"g": ["a", "b", "a"], "v": [1, 2, 3]})
print(df.groupby("g")["v"].sum().tolist())
```

Answer: `[4, 2]`

Practice

▼ **Exercise 1 — Honor roll**

Build a DataFrame with the names `Ana, Ben, Cy, Dee` and scores `91, 84, 77, 95`. Add a column `honors` that is True when the score is 90 or more. Print only the rows where `honors` is True.

▼ **Exercise 2 — Period averages**

Using the table from the groupby example, print the average score for each period, rounded to one decimal place with `.round(1)`.

Discussion

1. Why do you think pandas keeps the original index after filtering rows? When does that help, and when does it cause bugs?
2. A column of ages has one blank cell. Options: drop the row, fill with 0, fill with the average. Which would you choose for a survey, and why?
3. What question would you ask of a grade table before you trust a `groupby` average?

Key terms

DataFrame

A two-dimensional labeled table in pandas, with named columns that can each have a different type.

Series

A one-dimensional labeled array, such as a single column of a DataFrame.

Index

The labels for the rows of a DataFrame or Series. The default is 0, 1, 2, and so on.

read_csv

The pandas function that loads a CSV file (or text) into a DataFrame.

loc / iloc

`loc` selects by label; `iloc` selects by integer position.

NaN

Not a Number: pandas' marker for a missing value in a numeric column.

groupby

A method that splits rows into groups by the values of one or more columns, so you can summarize each group.

value_counts

A method that counts how often each distinct value appears in a column.

See all terms in the glossary →